

Contents

1	Alineamiento, similitud y búsqueda en bases de datos	1
1.1	De decodificar una secuencia a comparar dos	1
1.2	Alinear es optimizar	1
1.3	Needleman–Wunsch: el optimo global	3
1.3.1	Definición de la matriz $F(i, j)$ y condiciones de contorno	3
1.3.2	Ecuación de recurrencia	3
1.3.3	Traza completa resuelta: alinear ‘GAT’ contra ‘GC’	4
1.4	Smith–Waterman: el optimo local	6
1.4.1	Ecuación de recurrencia de Smith–Waterman	6
1.4.2	Traza resuelta: ‘CCGAT’ frente a ‘TTGAT’	6
1.5	Huecos y matrices de sustitución	7
1.5.1	Penalización de hueco afín (*Affine Gap Penalties*)	7
1.5.2	Matrices de sustitución de aminoácidos: PAM y BLOSUM	8
1.6	Bases de datos: identificadores, versiones y reproducibilidad	8
1.7	BLAST: una heurística, no una garantía	9
1.7.1	Estadística de Karlin–Altschul: puntuación bruta, Bit Score y E -value	10
1.8	La operación peligrosa: parámetros invertidos y números sin versión	11
1.9	Actividad de depuración: errores en Needleman–Wunsch	11
1.10	Síntesis operativa y tablas de diagnóstico	12
1.10.1	Tabla de diagnóstico de síntomas en alineamientos	12
1.10.2	Tabla de síntesis con preguntas de control	12
1.11	Glosario conceptual	13
1.12	Conexión con el curso y siguientes pasos	13
1.13	Fuentes y reproducibilidad	13
	Anexo práctico · Alinear, validar huecos y decidir con un E-value	15

Chapter 1

Alineamiento, similitud y búsqueda en bases de datos

1.1 De decodificar una secuencia a comparar dos

El capítulo anterior (TC-CH08) resolvió el modelado de una sola secuencia: dada una secuencia de ADN, clasificar su pertenencia a una clase (Naïve-Bayes) o reconstruir sus estados biológicos ocultos más probables (Viterbi).

Esencial

Este capítulo cambia radicalmente la pregunta científica: dadas **dos** secuencias biológicas, ¿cómo cuantificar su parecido y cuál es la correspondencia óptima posición a posición que explica ese parecido?

La técnica computacional que resuelve este problema —la **programación dinámica con matrices de puntuación y trazado hacia atrás (*backtracking*)**— es exactamente la misma maquinaria conceptual utilizada en el algoritmo de Viterbi:

- En Viterbi, cada celda optimiza el camino de estados ocultos a lo largo del tiempo para una sola secuencia.
- En alineamiento, cada celda optimiza la correspondencia entre los prefijos de dos secuencias distintas.

El alineamiento por programación dinámica reutiliza la técnica de retroceso de punteros vista en el algoritmo de Viterbi (TC-CH08), aplicada a comparar dos secuencias en vez de decodificar los estados ocultos de una sola: en ambos casos cada celda de una tabla guarda la mejor puntuación alcanzable y un puntero al origen de esa puntuación, y la solución completa se reconstruye retrocediendo esos punteros desde la celda final hasta el origen.

Esencial

La ruta **esencial** cubre la formulación matemática del alineamiento con penalizaciones de hueco, el alineamiento global (Needleman–Wunsch), el alineamiento local (Smith–Waterman), el modelo de hueco afín, la validación de cabeceras FASTA y el cálculo del *E*-value de BLAST; 120–140 minutos. La ruta de **estudio** profundiza en el análisis de sensibilidad de BLAST ante el tamaño de la base de datos, el impacto del enmascaramiento de regiones de baja complejidad, la actividad de depuración y el laboratorio práctico; 60–80 minutos adicionales.

1.2 Alinear es optimizar

Un **alineamiento** entre dos secuencias $X = x_1x_2\dots x_n$ e $Y = y_1y_2\dots y_m$ propone una superposición columna a columna insertando caracteres de espaciado o **huecos** (*gaps*, denotados por -) donde una secuencia presenta un nucleótido o aminoácido sin contrapartida en la otra.

Biológicamente, las coincidencias y sustituciones reflejan mutaciones puntuales por reemplazo, mientras que los huecos representan eventos de inserción o delección (**indels**).

Para comparar objetivamente distintos alineamientos posibles, se requiere un **sistema de puntuación** explícito:

- Recompensa por coincidencia exacta (*match*): $+m$ ($m > 0$).
- Penalización por discrepancia (*mismatch*): $-s$ o $s < 0$.
- Penalización lineal por hueco (*gap*): $-g$ o $g < 0$.

Un alineamiento propone correspondencias entre símbolos de dos secuencias e inserta huecos para representar inserciones o deleciones (indels). Compararlo con otros alineamientos requiere un sistema de puntuación explícito (recompensa por coincidencia, penalización por sustitución, penalización por hueco); el alineamiento óptimo es óptimo relativo a ese sistema de puntuación concreto, no de forma universal e independiente de los parámetros elegidos y del objetivo del análisis.

Alineamiento Global frente a Alineamiento Local.

- **Alineamiento Global (Needleman–Wunsch):** Alinea ambas secuencias de extremo a extremo ($1 \dots n$ contra $1 \dots m$). Es el método apropiado cuando las secuencias tienen longitudes similares y se espera homología a lo largo de toda su extensión.
- **Alineamiento Local (Smith–Waterman):** Busca la región o motivo subsecuencial de máxima similitud, ignorando los extremos flanqueantes no relacionados. Es indispensable para localizar dominios conservados dentro de proteínas multidominio o genes quiméricos.

El alineamiento global compara dos secuencias de extremo a extremo y es apropiado cuando su longitud y su homología son comparables en toda su extensión; el alineamiento local busca los mejores segmentos parciales y tolera flancos no relacionados, apropiado cuando solo una región interna de las secuencias es comparable.

Rigor conceptual: Identidad, Similitud y Homología. En bioinformática es imperativo no confundir tres conceptos clave:

1. **Identidad:** Porcentaje de posiciones alineadas que contienen exactamente el mismo residuo químico.
2. **Similitud:** Porcentaje o puntuación que contabiliza residuos con propiedades fisicoquímicas equivalentes (p. ej., aminoácidos hidrofóbicos o cargados).
3. **Homología:** Hipótesis biológica cualitativa sobre la existencia de un ancestro evolutivo común.

La homología es una propiedad dicotómica (dos secuencias son homólogas o no lo son; *no existe la «homología al 80%»*). Una identidad elevada constituye evidencia en favor de la homología, pero no la demuestra automáticamente.

Identidad (fracción de posiciones alineadas con el mismo símbolo), similitud (que añade criterios biológicos o químicos entre símbolos distintos pero relacionados) y homología (hipótesis de origen evolutivo común) no son sinónimos. Una identidad o similitud alta es evidencia hacia la homología, pero no la demuestra por sí sola; la homología no es un porcentaje —un par es o no es homólogo— y lo que varía en grado es la evidencia disponible.

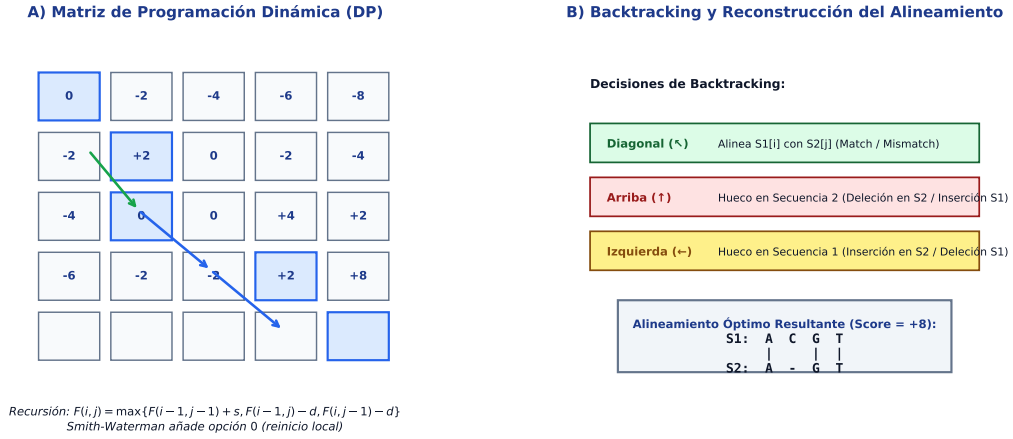


Figure 1.1: A) Matriz de Programación Dinámica evaluando los tres orígenes posibles (diagonal, arriba, izquierda) para alinear dos secuencias. B) Decisión de *backtracking* y alineamiento óptimo reconstruido de derecha a izquierda. Figura original adaptada de las presentaciones docentes.

1.3 Needleman–Wunsch: el óptimo global

El algoritmo de **Needleman–Wunsch** (1970) resuelve el problema del alineamiento global óptimo garantizado mediante programación dinámica.

1.3.1 Definición de la matriz $F(i, j)$ y condiciones de contorno

Sea $F(i, j)$ la puntuación del mejor alineamiento posible entre el prefijo $x_1 \dots x_i$ de longitud i y el prefijo $y_1 \dots y_j$ de longitud j .

Inicialización de fronteras: Alinear un prefijo contra una secuencia vacía solo puede lograrse insertando i (o j) huecos consecutivos:

$$F(0, 0) = 0, \quad F(i, 0) = i \cdot g, \quad F(0, j) = j \cdot g$$

En Needleman–Wunsch, $F(i, j)$ es la puntuación óptima de alinear los primeros i símbolos de x con los primeros j símbolos de y . La primera fila y la primera columna acumulan penalizaciones de hueco porque alinear un prefijo contra la cadena vacía solo puede hacerse insertando huecos. El retroceso de punteros parte de $F(n, m)$, que contiene la puntuación óptima del alineamiento global completo, y termina en $F(0, 0)$ reconstruyendo el alineamiento en orden inverso.

1.3.2 Ecuación de recurrencia

Cada celda interior $F(i, j)$ se calcula evaluando los tres orígenes posibles:

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & \text{(Diagonal: alinear } x_i \text{ con } y_j) \\ F(i-1, j) + g & \text{(Arriba: hueco en Y)} \\ F(i, j-1) + g & \text{(Izquierda: hueco en X)} \end{cases}$$

La recurrencia de Needleman–Wunsch es $F(i, j) = \max\{F(i-1, j-1) + s(x_i, y_j), F(i-1, j) + g, F(i, j-1) + g\}$, donde $s(x_i, y_j)$ es la recompensa por coincidencia o la penalización por sustitución según si x_i es igual a y_j , y g es la penalización por hueco.

Listing 1.1: Algoritmo formal en pseudocódigo para el Alineamiento Global de Needleman-Wunsch.

```

ALGORITMO: Needleman_Wunsch_Alineamiento_Global
ENTRADA:
- X: Secuencia 1 de longitud n
- Y: Secuencia 2 de longitud m
- s(a, b): Matriz de puntuacion (match = +m_score, mismatch = -s_score)
- g: Penalizacion lineal por hueco (g < 0)
SALIDA:
- (Score_Optimo, X_alineado, Y_alineado)
COMPLEJIDAD: Tiempo  $O(n * m)$ , Espacio DP  $O(n * m)$ 

1. INICIALIZACION DE FRONTERAS:
F[0, 0] <- 0
Para i desde 1 hasta n: F[i, 0] <- i * g
Para j desde 1 hasta m: F[0, j] <- j * g

2. LLENADO DE LA MATRIZ DE PROGRAMACION DINAMICA:
Para i desde 1 hasta n:
  Para j desde 1 hasta m:
    diag <- F[i - 1, j - 1] + s(X[i], Y[j])
    up <- F[i - 1, j] + g
    left <- F[i, j - 1] + g
    F[i, j] <- max(diag, up, left)

3. RECONSTRUCCION POR BACKTRACKING (Desde F[n, m] hasta F[0, 0]):
i <- n; j <- m
ali_X <- ""; ali_Y <- ""

Mientras i > 0 o j > 0:
  Si i > 0 y j > 0 y F[i, j] == F[i - 1, j - 1] + s(X[i], Y[j]) entonces:
    ali_X <- X[i] + ali_X; ali_Y <- Y[j] + ali_Y
    i <- i - 1; j <- j - 1
  Sino si i > 0 y F[i, j] == F[i - 1, j] + g entonces:
    ali_X <- X[i] + ali_X; ali_Y <- "-" + ali_Y
    i <- i - 1
  Sino:
    ali_X <- "-" + ali_X; ali_Y <- Y[j] + ali_Y
    j <- j - 1

4. RETORNO:
Retornar (F[n, m], ali_X, ali_Y)

```

1.3.3 Traza completa resuelta: alinear ‘GAT’ contra ‘GC’

Consideremos los parámetros estándar: coincidencia $m = +1$, discrepancia $s = -1$, hueco $g = -2$.

Construcción de la matriz $F(i, j)$ para $X = \text{‘GAT’}$ ($n = 3$) e $Y = \text{‘GC’}$ ($m = 2$):

$F(i, j)$	$\epsilon (j = 0)$	G ($j = 1$)	C ($j = 2$)
$\epsilon (i = 0)$	0	-2	-4
G ($i = 1$)	-2	+1 ($\nearrow$)	-1 ($\leftarrow$)
A ($i = 2$)	-4	-1 ($\uparrow$)	-1 ($\nearrow$)
T ($i = 3$)	-6	-3 ($\uparrow$)	-2 ($\nearrow, \uparrow$)

Trazado hacia atrás (*Backtracking*): Partiendo de la celda final $F(3, 2) = -2$:

1. En $F(3, 2) = -2$, se resuelve el empate eligiendo la diagonal $\nearrow \rightarrow$ alinea T con C ($s = -1$). Se pasa a $F(2, 1)$.
2. En $F(2, 1) = -1$, el origen es arriba $\uparrow \rightarrow$ alinea A con hueco - ($g = -2$). Se pasa a $F(1, 1)$.
3. En $F(1, 1) = +1$, el origen es diagonal $\nearrow \rightarrow$ alinea G con G ($m = +1$). Se pasa a $F(0, 0)$.

Alineamiento global resultante:

X: G A T
Y: G - C

Alineando GAT con GC (coincidencia +1, sustitución -1, hueco -2), la puntuación óptima de Needleman–Wunsch es -2 con alineamiento GAT / G-C: G-G coincide, A se alinea con un hueco, T-C no coincide. La celda final empatiza entre dos orígenes (diagonal y arriba); el retroceso elige diagonal de forma sistemática y reproducible. Se reproduce con `verification/chapter_results.sh nw_gat_gc`.

Verificación ortogonal de la puntuación: Calculando directamente sobre las columnas del alineamiento obtenido:

$$\text{Puntuación} = (+1 \text{ por G/G}) + (-2 \text{ por A/-}) + (-1 \text{ por T/C}) = 1 - 2 - 1 = -2$$

Recalcular la puntuación de GAT / G-C columna por columna sobre el alineamiento ya construido —sin repetir la matriz de programación dinámica— da de nuevo -2: una comprobación independiente que usa una propiedad distinta al procedimiento que produjo el resultado, no la misma matriz recalculada. Se reproduce con `verification/chapter_results.sh verify_alignment_score`.

Ejemplo de decisión de hueco vs discrepancia: ‘ACGTACGT’ vs ‘ACGTTTCGT’. Al alinear estas dos secuencias de longitud 8 con un único nucleótido distinto en la posición 5:

$$\text{Score sin huecos} = 7 \times (+1) + 1 \times (-1) = 6$$

Si se introdujera un hueco para evitar el mismatch, se incurriría en al menos dos penalizaciones de hueco ($-2 + -2 = -4$), obteniendo una puntuación estrictamente inferior. La selección depende exclusivamente del sistema de puntuación.

Alineando ACGTACGT con ACGTTTCGT (misma longitud, una posición distinta) con coincidencia +1, sustitución -1, hueco -2, el óptimo global no inserta ningún hueco: alinea símbolo a símbolo aceptando la única sustitución, con puntuación $7 \times (+1) + 1 \times (-1) = 6$. Introducir un hueco para evitar esa sustitución costaría al menos -2 por esa posición más el desplazamiento del resto: siempre peor que aceptar la sustitución a coste -1. La elección la determina la puntuación, no una preferencia algorítmica por evitar huecos. Se reproduce con `verification/chapter_results.sh nw_no_gap_preference`.

Complejidad computacional: El llenado de la matriz requiere evaluar $(n + 1)(m + 1)$ celdas a coste $O(1)$ por celda, resultando en una complejidad temporal de $O(n \cdot m)$ y memoria $O(n \cdot m)$.

Un empate entre dos o más orígenes en la recurrencia de Needleman–Wunsch significa que existe más de un alineamiento óptimo con la misma puntuación —información sobre el problema, no un error—; una implementación reproducible declara una regla determinista de desempate. Con penalización lineal por hueco, Needleman–Wunsch tiene complejidad $O(nm)$ en tiempo y memoria, ya que llena una tabla de $(n + 1)$ por $(m + 1)$ celdas en tiempo constante por celda.

1.4 Smith–Waterman: el óptimo local

El algoritmo de **Smith–Waterman** (1981) identifica el segmento subsecuencial de máxima similitud local mediante tres modificaciones fundamentales sobre Needleman–Wunsch:

1. **Fronteras en cero:** $H(i, 0) = 0$ y $H(0, j) = 0$ (no se acumulan penalizaciones iniciales).
2. **Piso en cero en la recurrencia:** Si todas las opciones son negativas, la celda toma el valor 0, reiniciando el alineamiento local.
3. **Inicio y fin del *backtracking*:** Comienza en la celda con el valor máximo absoluto de toda la tabla ($\max_{i,j} H(i, j)$) y finaliza en cuanto encuentra una celda con valor 0.

El alineamiento local (Smith–Waterman) añade un cuarto origen —el cero— a la recurrencia de programación dinámica, que reinicia la puntuación en cuanto acumular penalizaciones deja de compensar; inicializa los bordes a cero en vez de acumular huecos; y retrocede desde la celda de máxima puntuación de toda la tabla hasta la primera celda con valor cero, no hasta el origen. El resultado puede cubrir solo una región interna de las secuencias, no su extensión completa.

1.4.1 Ecuación de recurrencia de Smith–Waterman

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + s(x_i, y_j) \\ H(i-1, j) + g \\ H(i, j-1) + g \end{cases}$$

La recurrencia de Smith–Waterman es $H(i, j) = \max\{0, H(i-1, j-1) + s(x_i, y_j), H(i-1, j) + g, H(i, j-1) + g\}$: la única diferencia formal con Needleman–Wunsch es el término cero adicional, que impide que la puntuación baje de cero en ninguna celda.

1.4.2 Traza resuelta: ‘CCGAT’ frente a ‘TTGAT’

Con $m = +1, s = -1, g = -2$, el alineamiento local entre $X = \text{‘CCGAT’}$ e $Y = \text{‘TTGAT’}$ ignora por completo los flancos divergentes iniciales CC y TT (cuyas celdas se mantienen en 0) y acumula puntuación en el motivo idéntico común:

$$\begin{array}{rcl} X[3..5] & : & \text{G A T} \\ Y[3..5] & : & \text{G A T} \quad (\text{Puntuación máxima } H = 3) \end{array}$$

Alineando CCGAT con TTGAT (coincidencia +1, sustitución -1, hueco -2), Smith–Waterman recupera el alineamiento local GAT / GAT con puntuación 3, ignorando por completo los flancos CC y TT: un alineamiento global forzaría una correspondencia entre esos flancos no relacionados, reduciendo la puntuación total. Se reproduce con `verification/chapter_results.sh sw_ccgat_ttgat`.

Un mismo par de secuencias puede presentar varios máximos locales con igual puntuación bajo Smith–Waterman; cada uno es un resultado válido y debe reportarse con sus coordenadas. Comparar alineamientos locales únicamente por porcentaje de identidad favorece alineamientos muy cortos (alta identidad sobre pocas posiciones dice menos que identidad moderada sobre muchas posiciones); la longitud y la cobertura deben leerse junto a la identidad, no en su lugar.

Listing 1.2: Algoritmo formal en pseudocódigo para el Alineamiento Local de Smith–Waterman.

```
ALGORITMO: Smith_Waterman_Alineamiento_Local
ENTRADA:
```

```

- X: Secuencia 1 de longitud n, Y: Secuencia 2 de longitud m
- s(a, b): Matriz de similitud, g: Penalización de hueco (g < 0)
SALIDA:
- (Score_Maximo, Segmento_X_alineado, Segmento_Y_alineado)
COMPLEJIDAD: Tiempo O(n * m), Espacio DP O(n * m)

1. INICIALIZACION (Bordes a cero):
Para i desde 0 hasta n: H[i, 0] <- 0
Para j desde 0 hasta m: H[0, j] <- 0
max_score <- 0; (max_i, max_j) <- (0, 0)

2. LLENADO CON REINICIO EN CERO:
Para i desde 1 hasta n:
  Para j desde 1 hasta m:
    diag <- H[i - 1, j - 1] + s(X[i], Y[j])
    up <- H[i - 1, j] + g
    left <- H[i, j - 1] + g
    H[i, j] <- max(0, diag, up, left) # Terminó 0 permite reiniciar
    alineamiento

    Si H[i, j] > max_score entonces:
      max_score <- H[i, j]
      (max_i, max_j) <- (i, j)

3. BACKTRACKING LOCAL (Desde max_score hasta primer 0):
i <- max_i; j <- max_j
ali_X <- ""; ali_Y <- ""

Mientras H[i, j] > 0:
  Si H[i, j] == H[i - 1, j - 1] + s(X[i], Y[j]) entonces:
    ali_X <- X[i] + ali_X; ali_Y <- Y[j] + ali_Y
    i <- i - 1; j <- j - 1
  Sino si H[i, j] == H[i - 1, j] + g entonces:
    ali_X <- X[i] + ali_X; ali_Y <- "-" + ali_Y
    i <- i - 1
  Sino:
    ali_X <- "-" + ali_X; ali_Y <- Y[j] + ali_Y
    j <- j - 1

4. RETORNO:
Retornar (max_score, ali_X, ali_Y)

```

1.5 Huecos y matrices de sustitución

1.5.1 Penalización de hueco afín (*Affine Gap Penalties*)

Biológicamente, la probabilidad de que una delección o inserción involucre varios nucleótidos consecutivos en un solo evento es mucho mayor que la ocurrencia de múltiples delecciones independientes de 1 base.

El **modelo de penalización afín** penaliza fuertemente la apertura del hueco (g_o) y penaliza con menor intensidad cada posición adicional de extensión (g_e):

$$g(k) = g_o + (k - 1) \cdot g_e, \quad \text{con } g_o < 0, g_e < 0, \text{ y } |g_o| > |g_e|$$

El coste afín por hueco es $g(k) = g_o + (k - 1)g_e$ para un hueco de k posiciones, con apertura g_o y extensión g_e ambas negativas y $|g_o| > |g_e|$. Modela que un indel de varias posiciones suele

ser un único evento evolutivo, y hace que el modelo prefiera un hueco largo a varios huecos cortos que sumen la misma longitud total.

Cálculo de coste afín: Con apertura $g_o = -5$ y extensión $g_e = -1$, un hueco de longitud $k = 3$ cuesta:

$$g(3) = -5 + (3 - 1) \cdot (-1) = -5 - 2 = -7$$

(frente a -15 , que resultaría de abrir 3 huecos independientes).

Con apertura -5 y extensión -1 , un hueco afín de longitud 3 cuesta $g(3) = -5 + 2 \cdot (-1) = -7$, no -15 (que resultaría de tratar erróneamente las tres posiciones como tres aperturas independientes). El propósito del coste afín es que un hueco largo resulte más barato que varios huecos cortos abiertos por separado que sumen la misma longitud total. Se reproduce con `verification/chapter_results.sh affine_gap_cost 3`.

Validación de parámetros afines: Configurar $g_o = -1$ y $g_e = -5$ ($|g_o| < |g_e|$) es un error grave: haría que abrir un hueco nuevo fuera más barato que extender uno existente, destruyendo el supuesto biológico del modelo.

Unos parámetros de hueco afín con $g_o = -1$, $g_e = -5$ (extensión más costosa que apertura) son inválidos: harían que abrir un hueco nuevo en cada posición fuera más barato que extender uno existente, lo contrario de lo que el coste afín pretende modelar. Validar $|g_o| > |g_e|$ antes de alinear evita construir un modelo que penalice indels largos justo al revés de lo previsto. Se reproduce con `verification/chapter_results.sh validate_gap_params -1 -5`.

1.5.2 Matrices de sustitución de aminoácidos: PAM y BLOSUM

Para alinear proteínas no se utiliza una penalización uniforme, sino **matrices de sustitución log-odds**:

- **Matrices PAM (Point Accepted Mutation, Margaret Dayhoff):** Basadas en un modelo evolutivo explícito de mutaciones puntuales aceptadas sobre familias de proteínas muy cercanas. PAM1 equivale a un 1% de cambio; matrices como PAM250 modelan distancias evolutivas mayores mediante multiplicación matricial.
- **Matrices BLOSUM (Blocks Substitution Matrix, Henikoff & Henikoff):** Construidas directamente a partir de bloques conservados de proteínas homólogas observadas experimentalmente. El número indica el porcentaje de identidad umbral usado para agrupar secuencias:
 - **BLOSUM62:** Matriz estándar de propósito general para divergencias medias.
 - **BLOSUM45 / BLOSUM80:** BLOSUM45 para secuencias divergentes (alineamientos distantes); BLOSUM80 para secuencias muy parecidas.

Las matrices de sustitución de proteínas (PAM, BLOSUM) asignan una puntuación distinta a cada par de aminoácidos, estimada a partir de las sustituciones observadas en proteínas emparentadas, en vez de una recompensa fija por coincidencia o sustitución. La elección de matriz y penalizaciones de hueco depende del tipo de secuencia, la divergencia esperada y el objetivo del análisis; las puntuaciones brutas de alineamientos calculados con esquemas distintos no son directamente comparables entre sí.

1.6 Bases de datos: identificadores, versiones y reproducibilidad

Una búsqueda de secuencias solo es científicamente reproducible si se registran los identificadores unívocos y sus versiones.

Estructura formal del formato FASTA: La línea de cabecera debe comenzar estrictamente con el carácter > seguido inmediatamente por el identificador o número de acceso (sin espacios en blanco intermedios), y opcionalmente una descripción tras el primer espacio:

- **Válido:** >NC_000913.3 Escherichia coli str. K-12 complete genome
- **Inválido:** > descripcion sin accesion (espacio tras >, número de acceso vacío).
- **Inválido:** NC_000913.3 secuencia (falta el carácter >).

Las bases de datos biológicas identifican cada registro con un número de acceso, habitualmente versionado, porque un mismo identificador sin versión puede referirse a contenidos distintos en fechas diferentes. El resultado de una búsqueda solo es reproducible si registra la secuencia de consulta y su procedencia, el programa y su versión, la base de datos y su versión o fecha, y los parámetros empleados; la ausencia de cualquiera de esas piezas rompe la reproducibilidad, aunque la puntuación calculada sea correcta.

Una cabecera FASTA reproducible comienza con > seguido del identificador o número de acceso sin espacio previo, y opcionalmente una descripción tras el primer espacio. La cabecera >NC_000913.3 Escherichia coli str. K-12 chromosome, complete genome es válida con número de acceso NC_000913.3; una línea sin > inicial, o con > seguido inmediatamente de un espacio (número de acceso vacío), es inválida y debe rechazarse antes de procesar el fichero, no después. Se reproduce con `verification/chapter_results.sh validate_fasta_header`.

El riesgo de la propagación circular de anotaciones: En los repositorios biológicos públicos, una gran proporción de anotaciones funcionales proceden de inferencias computacionales automáticas basadas en similitud de secuencia y no de caracterizaciones bioquímicas directas. Si un registro anotado automáticamente con una función no confirmada se utiliza como referencia para anotar nuevas secuencias, el error se propaga en cascada.

Una anotación funcional asociada a un registro de base de datos no es un hecho demostrado: las descripciones se propagan entre registros por similitud de secuencia, heredando y a veces perpetuando errores de anotaciones previas sin nueva evidencia experimental. Atribuir función a partir de un alineamiento exige una cadena de evidencia completa (procedencia, programa, parámetros, acierto, taxonomía, curación, alternativas consideradas); la fuente primaria y la validación experimental tienen más peso que una descripción heredada del primer acierto.

1.7 BLAST: una heurística, no una garantía

Ejecutar el algoritmo exacto de Smith–Waterman sobre una base de datos genómica con miles de millones de nucleótidos requeriría días de cómputo exhaustivo ($O(N \cdot M)$).

BLAST (*Basic Local Alignment Search Tool*, Altschul et al., 1990) es una aproximación ****heurística**** en 3 fases:

1. **Sembrado (*Seeding*)**: Encuentra coincidencias exactas de palabras cortas (W -meros; $W = 11$ en nucleótidos, $W = 3$ en proteínas).
2. **Extensión (*Extension*)**: Extiende la alineación a partir de la semilla en ambas direcciones mientras la puntuación acumulada no decaiga por debajo de un umbral X .
3. **Evaluación estadística (*HSP Evaluation*)**: Calcula la significación estadística del segmento de alta puntuación (*High-scoring Segment Pair*, HSP).

BLAST evita llenar una matriz de programación dinámica completa contra cada secuencia de una base de datos mediante una heurística en tres fases: busca palabras cortas exactas de la

consulta en la base de datos (semillas), extiende cada coincidencia mientras la puntuación mejore, y evalúa estadísticamente los pares de segmentos de alta puntuación resultantes. Es mucho más rápido que la programación dinámica exhaustiva, pero no garantiza encontrar el óptimo que hallaría Smith–Waterman: una coincidencia real sin una semilla exacta suficientemente larga puede pasar desapercibida.

1.7.1 Estadística de Karlin–Altschul: puntuación bruta, Bit Score y E -value

Puntuación bruta (S), puntuación normalizada (*Bit Score*) y significación (E -value). En BLAST se distinguen tres magnitudes estadísticas fundamentales:

- **Puntuación bruta (S):** suma de aciertos, desajustes y penalizaciones según la matriz empleada. No permite comparar búsquedas con matrices distintas.
- **Puntuación en bits (*Bit Score*, S'):** puntuación normalizada independiente del sistema de puntuación ($S' = \frac{\lambda S - \ln K}{\ln 2}$). Dos alineamientos con el mismo S' representan la misma calidad intrínseca aunque procedan de esquemas distintos.
- **Valor esperado (E -value):** número de alineamientos con puntuación $\geq S$ esperados puramente por azar en un espacio de búsqueda de tamaño $m \times n$:

$$E = K \cdot m \cdot n \cdot e^{-\lambda S} = m \cdot n \cdot 2^{-S'}$$

Regla de lectura inferencial: Un E -value bajo indica que el alineamiento es extremadamente improbable por puro azar, pero no equivale a una «probabilidad de homología». La homología es una hipótesis biológica de ancestría común que exige corroborar cobertura, dominios y coherencia taxonómica. donde K y λ son parámetros estadísticos calibrados según el sistema de puntuación, m es la longitud de la secuencia de consulta y n es el tamaño total de la base de datos en residuos.

El E -value de BLAST se estima como $E \approx K \cdot m \cdot n \cdot e^{-\lambda S}$, con K y λ parámetros estadísticos calibrados del sistema de puntuación, m la longitud de la consulta, n el tamaño de la base de datos y S la puntuación bruta del acierto. No es la probabilidad de que la secuencia sea homóloga ni de que su anotación sea correcta: es una estimación de aciertos esperados por azar para ese tamaño de búsqueda.

Análisis de sensibilidad y dependencia del tamaño n : Consideremos $K = 0.1, \lambda = 0.5, m = 20, n = 10^6$:

- Para $S = 40$: $E = 0.1 \times 20 \times 10^6 \times e^{-0.5 \times 40} = 2 \times 10^6 \times e^{-20} \approx \mathbf{0.004122}$ (acierto muy significativo).
- Si se duplica la base de datos a $n = 2 \times 10^6$ manteniendo la misma puntuación $S = 40$: $E = 2 \times 0.004122 = \mathbf{0.008245}$. El mismo alineamiento pierde significación puramente porque el espacio de búsqueda es mayor.
- Sensibilidad exponencial ante S : Para $S = 30 \implies E \approx \mathbf{0.611805}$ (no significativo, esperable por azar). Para $S = 50 \implies E \approx \mathbf{0.000028}$ (altamente significativo).

Con $K = 0.1, \lambda = 0.5, m = 20, n = 1000000$ y $S = 40$, el E -value de BLAST es aproximadamente 0.004122: un acierto significativo, con menos de 0.005 coincidencias igual de buenas esperadas por azar en una búsqueda de ese tamaño. Se reproduce con `verification/chapter_results.sh blast_evalue_calc 0.1 0.5 20 1000000 40`.

Con $K = 0.1, \lambda = 0.5, m = 20, S = 40$ fijos, duplicar el tamaño de la base de datos de $n = 1000000$ a $n = 2000000$ duplica también el E -value (de 0.004122 a 0.008245): el mismo

acierto se vuelve menos significativo puramente porque la base de datos creció, sin ningún cambio en la calidad del alineamiento en sí. Se reproduce con `verification/chapter_results.sh blast_evalue_scaling`.

Con $K = 0.1$, $\lambda = 0.5$, $m = 20$, $n = 1000000$ fijos, una puntuación $S = 30$ da $E \approx 0.611805$ (no significativo, más de medio acierto igual de bueno esperado por azar), mientras que $S = 50$ da $E \approx 0.000028$ (altamente significativo): pequeñas diferencias de puntuación cerca del umbral de significación cambian el E-value en varios órdenes de magnitud. Se reproduce con `verification/chapter_results.sh blast_evalue_sensitivity`.

Ficha técnica obligatoria para búsquedas reproducibles con BLAST: Todo informe o pipeline científico que utilice BLAST debe registrar explícitamente: (1) identificador y hash de la consulta (`*query*`); (2) programa, tarea y versión exacta; (3) base de datos con fecha de congelación y versión/parche; (4) matriz de sustitución y penalizaciones afines (g_o, g_e); (5) tamaño de palabra (`word_size`) y umbral de siembra; (6) política de enmascaramiento (`*masking*` duro frente a suave); (7) umbral de corte de significación (E -value threshold).

Enmascaramiento de baja complejidad (`*Masking*`): Regiones ricas en repeticiones simples (p. ej., poli-A o regiones ricas en prolina) generan semillas masivas por sesgo composicional. El enmascaramiento (con DUST o SEG) sustituye estas regiones por N o X para evitar falsos positivos espurios.

Las regiones de composición sesgada o baja complejidad producen palabras semilla muy frecuentes que reflejan composición compartida, no parentesco específico. El enmascaramiento las excluye de la búsqueda de semillas reduciendo falsos positivos, pero puede ocultar una señal real si se aplica sin revisión; se conservan tanto los resultados con enmascaramiento como sin él como escenarios distintos en vez de elegir el que dé el E-value más favorable.

1.8 La operación peligrosa: parámetros invertidos y números sin versión

Los dos errores más dañinos en pipelines de alineamiento son:

1. **Penalizaciones de hueco afín invertidas** ($|g_o| < |g_e|$): Produce fragmentación patológica en cientos de huecos de 1 nucleótido.
2. **Referencias a bases de datos sin versión:** Almacenar únicamente el nombre del genoma sin fecha de congelación ni número de parche rompe la reproducibilidad científica.

Protocolo preventivo en 3 pasos:

1. **Aserción de apertura/extensión:** Comprobar $|g_o| > |g_e|$ antes de instanciar la matriz.
2. **Verificación sintáctica de FASTA:** Comprobar que la primera línea comience por `>` seguido inmediatamente de caracteres no vacíos.
3. **Congelación de metadatos:** Registrar siempre el número de versión de la base de datos y la semilla en cualquier informe de BLAST.

1.9 Actividad de depuración: errores en Needleman–Wunsch

Analice el siguiente fragmento con tres errores conceptuales en la inicialización y recurrencia de Needleman–Wunsch:

Listing 1.3: Fragmento con tres errores en la implementacion de Needleman–Wunsch.

```

needleman_wunsch(seq1, seq2, match, mismatch, gap):
  # Fallo 1: inicializacion de fronteras en 0 en lugar de penalizaciones
  acumuladas
  para i de 0 a len(seq1): F[i][0] <- 0
  para j de 0 a len(seq2): F[0][j] <- 0

  para i de 1 a len(seq1):
    para j de 1 a len(seq2):
      # Fallo 2: se suma gap a la diagonal en lugar de evaluar match/mismatch
      opc_diag <- F[i-1][j-1] + gap
      opc_arriba <- F[i-1][j] + gap
      opc_izq <- F[i][j-1] + gap
      F[i][j] <- max(opc_diag, opc_arriba, opc_izq)
      # Fallo 3: no se guardan punteros de direccion para el backtracking

  devolver F[len(seq1)][len(seq2)]

```

1.10 Síntesis operativa y tablas de diagnóstico

1.10.1 Tabla de diagnóstico de síntomas en alineamientos

Síntoma observado	Causa probable	Comprobación y corrección
Alineamiento global introduce decenas de huecos de 1 base	Parámetros afines invertidos ($ g_o < g_e $)	Ajustar $ g_o > g_e $ (p. ej., $g_o = -10, g_e = -1$)
Smith–Waterman devuelve una tabla llena de ceros	Recompensa por match demasiado baja frente al mismatch	Asegurar que la puntuación esperada por residuo sea negativa
El E -value de un hit sube tras actualizar la base de datos	Aumento legítimo del tamaño n de la base	Es el comportamiento esperado de la estadística de Karlin
Cabecera FASTA ignorada o parseada como vacía	Espacio entre > y el ID (> ID)	Eliminar espacios en blanco tras >

1.10.2 Tabla de síntesis con preguntas de control

Tipo de problema	Herramienta recomendada	Pregunta de control
Homología global en secuencias de igual longitud	Needleman–Wunsch	¿Se acumulan penalizaciones de gap en las fronteras?
Detección de dominios y motivos locales	Smith–Waterman	¿Se incluye el piso en cero y se inicia en el máximo?
Búsqueda masiva en bases de datos genómicas	BLAST (heurística)	¿Se ha registrado el E -value, la versión y el enmascaramiento?

1.11 Glosario conceptual

- **Alineamiento global:** optimización de extremo a extremo (Needleman–Wunsch).
- **Alineamiento local:** búsqueda del motivo de máxima similitud (Smith–Waterman).
- **Indel:** evento mutacional de inserción o delección representado por un gap.
- **Hueco afín:** modelo con penalización diferenciada para apertura (g_o) y extensión (g_e).
- **Matriz PAM:** matriz de sustitución basada en mutaciones puntuales aceptadas (Dayhoff).
- **Matriz BLOSUM:** matriz de sustitución basada en bloques de proteínas observadas.
- **BLAST:** heurística de búsqueda rápida basada en semillas W -meros y extensión.
- **E-value:** número de aciertos esperados puramente por azar ($Kmne^{-\lambda S}$).
- **Homología:** relación evolutiva cualitativa derivada de un ancestro común.
- **Backtracking:** reconstrucción del alineamiento retrocediendo punteros desde el óptimo.

1.12 Conexión con el curso y siguientes pasos

El alineamiento por programación dinámica y la decodificación de Viterbi (TC-CH08) comparten dos ideas metodológicas: la programación dinámica con retroceso de punteros para reconstruir una solución completa a partir de una tabla de puntuaciones parciales, y la verificación independiente de un resultado mediante una vía distinta a la que lo produjo. Una puntuación de alineamiento o un E-value describe la similitud de secuencia bajo un modelo estadístico declarado; no demuestra homología, función compartida ni relevancia clínica sin una cadena de evidencia adicional. Las redes neuronales (TC-CH10) abordan el mismo problema de comparación y clasificación de secuencias aprendiendo representaciones a partir de datos, en vez de partir de un esquema de puntuación fijo definido a mano.

1.13 Fuentes y reproducibilidad

Este capítulo se fundamenta en las presentaciones docentes (20251028_alg_3.pptx y 20251125_bioinfo_5.pptx Álvaro Serrano Navarro), y en los trabajos clásicos de Needleman & Wunsch (1970), Smith & Waterman (1981) y Altschul et al. (1990). Todos los resultados numéricos se validan determinísticamente mediante `verification/chapter_results.sh`.

Anexo práctico · Alinear, validar huecos y decidir con un E-value

Pregunta, producto y separación de materiales

¿Puede completar y retroceder a mano una matriz de alineamiento global y una local, calcular correctamente el coste de un hueco afín y detectar parámetros de hueco inválidos, interpretar un E-value de BLAST y cómo cambia con el tamaño de la base de datos, y validar el formato mínimo reproducible de una cabecera de base de datos? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual contiene: el alineamiento global manual de ACT frente a AGT contrastado con el script, el alineamiento local manual de AACGTAA frente a TTCGTTT, el coste de un hueco afín válido y la detección de uno inválido, el E-value de un hit con dos tamaños de base de datos distintos, la validación de una cabecera FASTA válida y de dos inválidas, y los dos controles de caso límite, todo en `notas/respuestas.tsv`.

El anexo es autónomo e independiente: no depende de ningún fichero externo. Utiliza `practice_assets/create_workspace.sh` para crear el espacio de trabajo del alumno y `verification/chapter_results.sh` como herramienta de cómputo y auditoría.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Preparar espacio de trabajo	5	Directorio creado, herramienta comprobada
2 · Alineamiento global (Needleman–Wunsch)	15–20	Matriz manual contrastada con script
3 · Alineamiento local (Smith–Waterman)	15–20	Matriz manual y motivo recuperado
4 · Huecos afines	10–15	Coste calculado y parámetros inválidos detectados
5 · E-value y bases de datos	15–20	Interpretación de significancia y sensibilidad
6 · Cabeceras FASTA	10	Formato reproducible validado
7 · Controles de robustez	10	Dos casos límite verificados
8 · Verificación y empaquetado	5	Comprobación con <code>DELIVERY_OK</code>

La ruta única de este anexo (no hay separación esencial/estudio) suma 80–110 minutos y produce evidencia comprobable en cada bloque.

1 · Preparar el espacio de trabajo

Listing 1.4: Comprobación de entorno y espacio de trabajo.

```
bash --version
```

```
chmod +x practice_assets/create_workspace.sh
./practice_assets/create_workspace.sh TC09_entrega
cd TC09_entrega
```

2 · Alineamiento global: Needleman–Wunsch

Antes de ejecutar nada, complete a mano la matriz F para alinear ACT con AGT (coincidencia +1, sustitución −1, hueco −2, la misma recurrencia de la sección de Needleman–Wunsch) y anote la puntuación óptima y el alineamiento. Después contraste:

Listing 1.5: Contrastar el alineamiento global manual.

```
bash herramienta/chapter_results.sh needleman_wunsch ACT AGT 1 -1 -2
```

Alineando ACT con AGT (coincidencia +1, sustitución −1, hueco −2), la puntuación óptima de Needleman–Wunsch es 1: A-A coincide, C-G sustituye, T-T coincide, sin ningún hueco ($1 - 1 + 1 = 1$); el alineamiento es ACT / AGT sin desplazamiento alguno. Se reproduce con `verification/chapter_results.sh needleman_wunsch ACT AGT 1 -1 -2`.

3 · Alineamiento local: Smith–Waterman

Alinee ahora, de forma local, AACGTAA con TTCGTTT —mismos parámetros que el bloque anterior—. Prediga primero qué motivo compartido debería recuperar el algoritmo antes de ejecutar:

Listing 1.6: Contrastar el alineamiento local manual.

```
bash herramienta/chapter_results.sh smith_waterman AACGTAA TTCGTTT 1 -1 -2
```

Alineando localmente AACGTAA con TTCGTTT (coincidencia +1, sustitución −1, hueco −2), Smith–Waterman recupera el motivo compartido CGT / CGT con puntuación 3, ignorando los flancos AA/TT al inicio y AA/TT al final de cada secuencia respectivamente. Se reproduce con `verification/chapter_results.sh smith_waterman AACGTAA TTCGTTT 1 -1 -2`.

Compruebe además ese resultado por la vía independiente de la sección de Needleman–Wunsch —recalculando el score carácter a carácter sobre el alineamiento ya obtenido, sin repetir la matriz—:

Listing 1.7: Comprobacion independiente del score local.

```
bash herramienta/chapter_results.sh recompute_alignment_score CGT CGT 1 -1 -2
```

4 · Huecos afines

Calcule a mano el coste de un hueco afín de longitud 4 con apertura −6 y extensión −2, y después una fila propuesta por una IA con apertura −1 y extensión −5 —anote por qué esa segunda fila es inválida antes de ejecutar nada—:

Listing 1.8: Coste de hueco afín y detección de parámetros inválidos.

```
bash herramienta/chapter_results.sh affine_gap_cost 4 -6 -2
bash herramienta/chapter_results.sh validate_affine_gap_params -1 -5
```

Un hueco afín de longitud 4 con apertura −6 y extensión −2 cuesta $g(4) = -6 + 3 \cdot (-2) = -12$. Los parámetros apertura −1, extensión −5 son inválidos porque $|-1| <$

|−5|: harían más barato abrir un hueco nuevo en cada posición que prolongar uno existente, lo contrario de lo que un modelo de hueco afín debe garantizar. Se reproduce con `verification/chapter_results.sh affine_gap_cost 4 -6 -2` y `verification/chapter_results.sh validate_affine_gap_params -1 -5`.

5 · E-value y tamaño de la base de datos

Con $K = 0,08$, $\lambda = 0,4$, consulta $m = 25$ y score $S = 35$, calcule el E-value contra una base de $n = 2\,000\,000$ secuencias y decida si el hit es significativo. Después repita contra una base 5 veces mayor ($n = 10\,000\,000$) sin cambiar nada más, y anote qué cambia y por qué:

Listing 1.9: E-value con dos tamanos de base de datos.

```
bash herramienta/chapter_results.sh blast_evalue 0.08 0.4 25 2000000 35
bash herramienta/chapter_results.sh blast_evalue 0.08 0.4 25 10000000 35
```

Con $K = 0.08$, $\lambda = 0.4$, $m = 25$, $S = 35$, el E-value contra $n = 2\,000\,000$ es ≈ 3.326115 -no significativo, más de tres coincidencias igual de buenas esperadas por azar-; contra una base 5 veces mayor, $n = 10\,000\,000$, el E-value crece proporcionalmente a ≈ 16.630574 , con el mismo score y el mismo hit: crecer la base de búsqueda nunca hace un hit más significativo, y puede volver no significativo lo que en una base pequeña parecía prometedor. Se reproduce con `verification/chapter_results.sh blast_evalue 0.08 0.4 25 2000000 35` y `verification/chapter_results.sh blast_evalue 0.08 0.4 25 10000000 35`.

6 · Cabeceras FASTA

Valide las tres cabeceras siguientes y anote, para cada una, si es válida y por qué:

Listing 1.10: Validar el formato reproducible de una cabecera.

```
bash herramienta/chapter_results.sh validate_fasta_header ">seq_A"
bash herramienta/chapter_results.sh validate_fasta_header "> descripcion sin
  accesion"
bash herramienta/chapter_results.sh validate_fasta_header "seq_A sin corchete
  inicial"
```

`>seq_A` es una cabecera FASTA válida (accesión `seq_A`, sin descripción); `> descripcion sin accesion` es inválida porque el primer carácter tras `>` es un espacio (accesión vacía); `seq_A sin corchete inicial` es inválida porque no empieza por `>`. Las tres decisiones deben tomarse antes de procesar el fichero como si fuera una secuencia válida, no después de un error de más adelante en la tubería. Se reproduce con `verification/chapter_results.sh validate_fasta_header` sobre las tres cabeceras.

7 · Controles de robustez

Listing 1.11: Comprobar casos limite.

```
bash herramienta/chapter_results.sh needleman_wunsch "" "" 1 -1 -2
bash herramienta/chapter_results.sh smith_waterman A A 1 -1 -2
```

Alinear dos secuencias vacías con Needleman–Wunsch no falla: la única celda de la matriz es $F(0, 0) = 0$, y el alineamiento es vacío frente a vacío -puntuación 0, sin coincidencias, sustituciones ni huecos-. Alinear un único símbolo idéntico con Smith–Waterman (A frente a A) da el

caso más simple posible: puntuación 1, un único origen diagonal, sin necesidad de ningún retroceso multi-paso. Se reproduce con `verification/chapter_results.sh needleman_wunsch` y `verification/chapter_results.sh smith_waterman` sobre esos casos.

8 · Verificar la entrega

Listing 1.12: Comprobar que la entrega esta completa antes de empaquetar.

```
cd ..
../practice_assets/check_practice_delivery.sh TC09_entrega
tar -czf TC09_entrega.tar.gz TC09_entrega
sha256sum TC09_entrega.tar.gz
```

`check_practice_delivery.sh` verifica, con Bash y GNU Coreutils exclusivamente, que `notas/respuestas.tsv` existe con la cabecera esperada y al menos una fila rellena, y que `herramienta/chapter_results.sh` está presente y es ejecutable.

Rúbrica de calificación ponderada

La evaluación de este anexo pondera la verificación técnica, el cálculo de alineamientos y la defensa conceptual sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica y verificador (DELIVERY_OK):** Integridad del paquete comprimido `.tar.gz`, formato requerido de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.
- **2,5 puntos · Matrices de programación dinámica (NW y SW):** Construcción manual de tablas de puntuación, condiciones de contorno y reconstrucción por retroceso (*backtracking*).
- **2,5 puntos · Modelo afín, estadística de Karlin–Altschul y reproducibilidad:** Cálculo de penalizaciones afines, análisis de sensibilidad del *E*-value ante cambios en la base de datos y validación estricta de cabeceras FASTA.
- **2,0 puntos · Defensa conceptual y cautela inferencial:** Justificación rigurosa (100–150 palabras) sobre por qué la significación estadística (*E*-value) y la similitud no equivalen automáticamente a homología ni a función biológica demostrada.

Nota didáctica: La obtención de `DELIVERY_OK` acredita la ejecución mecánica de los scripts, pero no convalida la derivación manual de matrices ni el juicio crítico sobre los resultados de BLAST.

Lista de autocorrección

- | | |
|--|---|
| • Mi matriz manual de ACT/AGT coincidió con el script. | ida. |
| • Recuperé el motivo CGT en el alineamiento local y comprobé su score. | • Registré cómo cambia el E-value al multiplicar por 5 el tamaño de la base de datos. |
| • Calculé correctamente el coste del hueco afín de longitud 4 y detecté la fila invál- | • Clasifiqué correctamente las tres cabeceras FASTA como válidas o in- |

válidas y por qué.

- Los dos controles de caso límite están anotados sin fallos.
- Puedo explicar por qué un E-value bajo no equivale a homología demostrada.
- `check_practice_delivery.sh` termina con `DELIVERY_OK`.

Defensa conceptual

En 100–150 palabras, explique por qué duplicar el tamaño de una base de datos duplica el E-value de un hit con la misma puntuación, y qué tendría que cambiar en el score de ese hit para compensar exactamente ese aumento y mantener el mismo E-value.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes de bioinformática (20251125_bioinfo_5.pptx) y se valida al 100% mediante `verification/chapter_results.sh`.